AI × SOFTWARE ENGINEERING

Ohjelmistokehitys autonomisten agenttien aikakaudella

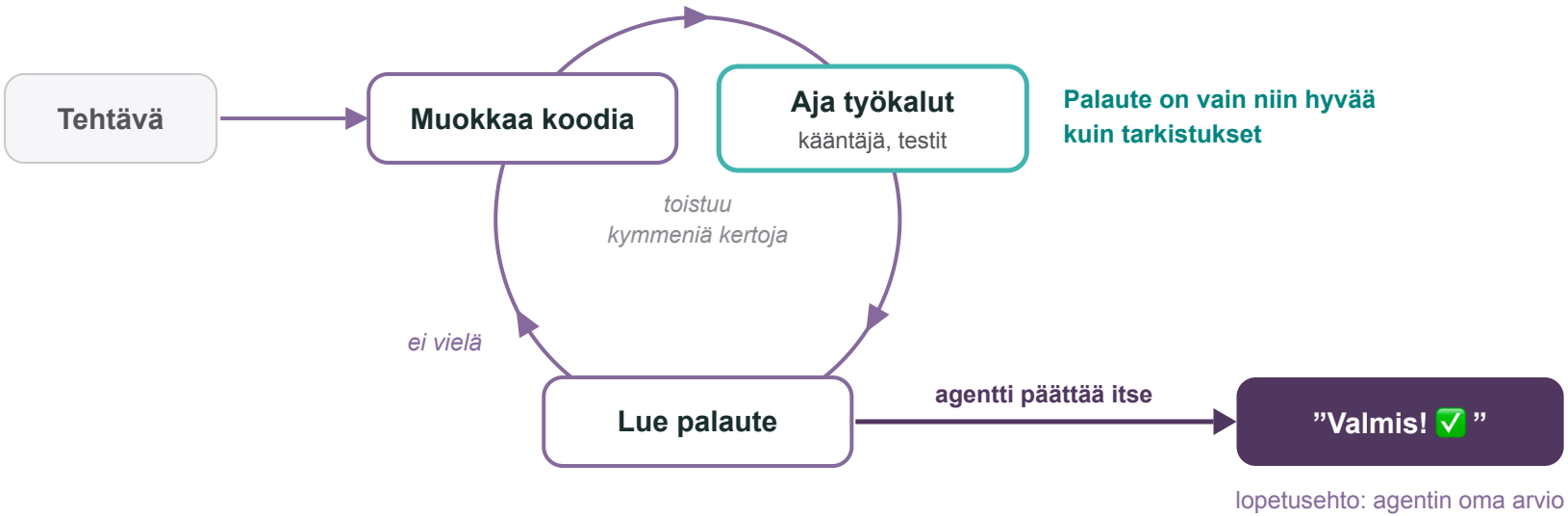
Miten annamme tekoälylle enemmän autonomiaa menettämättä kontrollia?

Joonas Laukka · Senior Consultant, Loihde

Määrittele järjestelmä, eristä agentti, päästä se irti.

LOIHDE

Agentti työskentelee silmukassa



Mistä tiedämme, että agentti on valmis?

HAVAINNOLLISTUS – EI OIKEAN AJON TULOSTE

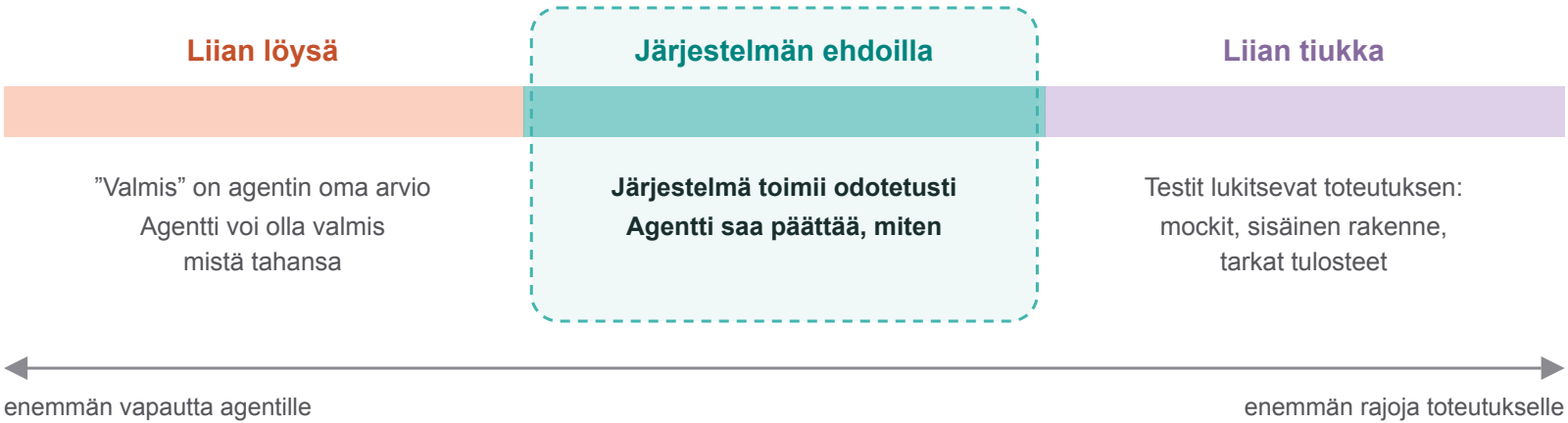
```
agent› Toteutin alennuslogiikan.
agent› Lisäsin testit ja kaikki menevät
läpi.
agent› Valmis! ✅
```

Mikä tässä voi mennä pieleen?

- **Sama väärinkäsitys kahdesti.** Agentti kirjoitti sekä toteutuksen että testit, joten väärinkäsitys päätyy molempiin, ja testit menevät silti läpi.
- **Testejä voi muuttaa.** Jos agentti saa muokata testejä, se voi saada tarkistuksen läpi muuttamalla tarkistusta eikä koodia.
- **Agentti voi erehtyä omasta valmiudestaan.** Se voi jättää testin ajamatta tai selittää epäonnistumisen pois, ja raportti näyttää silti samalta.

"Valmis! ✅ " on **väite**. Hyväksymiskriteeri on jotain, minkä päätimme **ennen** kuin delegoimme.

Rajataan ratkaisua jättäen agentille liikkumavaraa



Tavoite: **tarpeeksi tiukka**, että järjestelmä toimii odotetusti, ja **tarpeeksi löysä**, että agentti saa päättää, miten.

Deterministisiä tarkistuksia on monenlaisia

Tyypit — ms
Pakottavat sopimuksen ja ohjaavat toteutusta
`demossa: TypeScript`

Lint — ms
Kielletyt rakenteet ja tyyli

Yksikkötestit — s
Yksittäisen funktion toiminta

Ominaisuustestit — s
Käytös, johon muut järjestelmät luottavat
`demossa: fast-check`

Hyväksymistestit — s
Täyttyykö tarve, jonka vuoksi työ tehtiin
`demossa: Gherkin`

Golden-testit — s
Tuloste vastaa hyväksyttyä

Arkkitehtuuritestit — s
Kuka saa riippua kenestä

Kontraktitestit — min
Palveluiden välinen sopimus

Integraatiotestit — min
Komponentit toimivat yhdessä

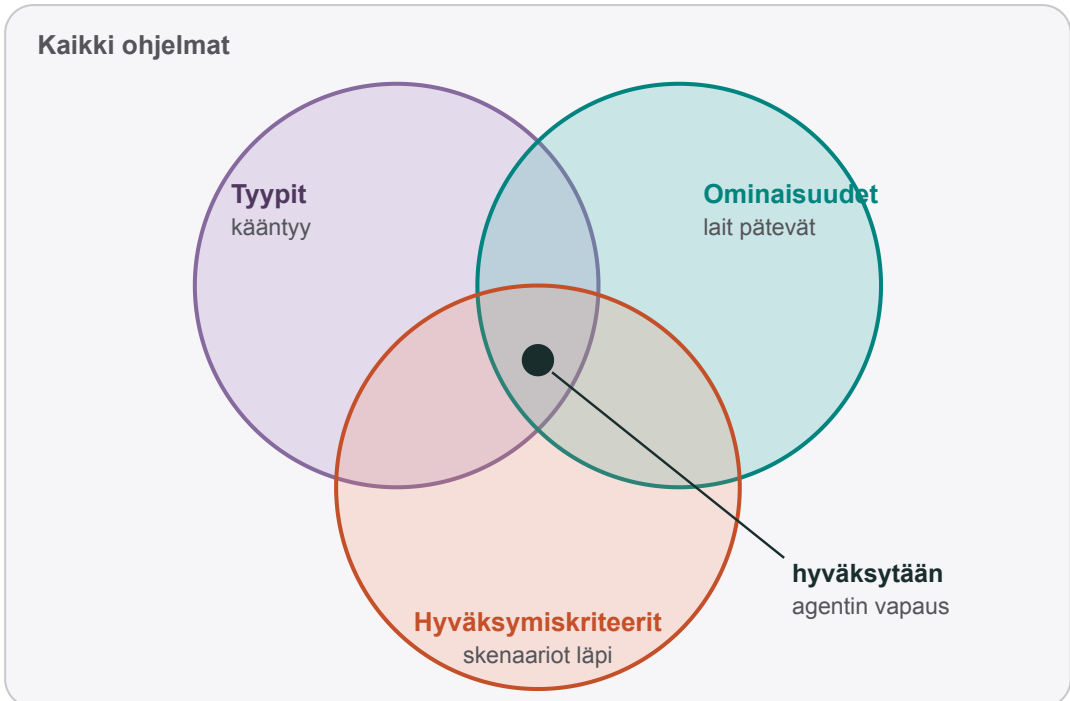
Tietoturvaskannaus — min
Tunnetut haavoittuvuudet ja vaaralliset rakenteet

E2E / UI-testit — min–h
Koko polku käyttäjän silmin

Fuzzing — min–h
Kaatuuko satunnaisella syötteellä

Tänään **kolme**, jotka kirjoittavat järjestelmän **lait**. Ne kuvaavat järjestelmää eivätkä koodia, joten ne pysyvät totena, vaikka toteutus vaihtuu.

Rajataan hyväksyttävien ohjelmien joukkoa



- Jokainen tarkistus **rajaa** hyväksyttävien ohjelmien joukkoa.
- Hyväksytään vain **leikkaus**: kääntyy, lait pätevät ja skenaariot menevät läpi.
- Leikkauksessa on yhä **monta** ratkaisua. Se on agentin vapaus.
- Ihminen vastaa siitä, **osuvatko rajat** tarkoitukseen.

Käsitteellinen malli hyväksynnästä. Ei väite siitä, että mallin generointia rajattaisiin muodollisesti.

Esimerkki: alennus verkkokaupan kassalla

KEKSITTY ESIMERKKI

Ostoskori	100,00 €
`AUTUMN20`	✓ −20 %
`SUMMER10`	✗ vanhentunut
Maksettava	**80,00 €**

Mitä esimerkissä on

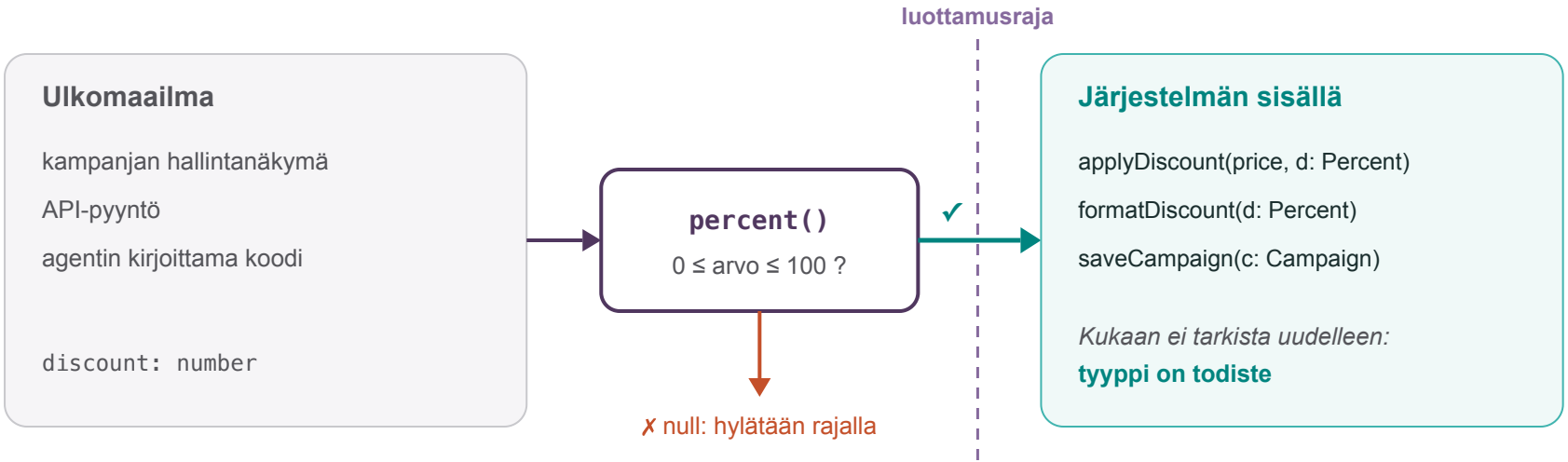
- **Hinta ja alennus.** `Money` sentteinä, `Percent` välillä 0–100.
- **Yksi funktio.** `applyDiscount(price, discount)`
- **Kampanjakoodit.** Voimassa oleva koodi alentaa hintaa, vanhentunut hylätään.

Jätetty pois: verot, valuutat, tuotekohtaiset alennukset, palautukset.

Esimerkki on **tarkoituksella äärimmäisen yksinkertainen**. Se näyttää mekanismin. Oikeassa työssä tarkistusten määrä riippuu muutoksen riskistä.

Tyypit

1 / 3 · Tyyppi kantaa todisteen



Tarkistus tehdään **kerran rajalla**. Tyyppi kantaa todisteen eteenpäin.

Ajatus tunnetaan nimellä ”Parse, don't validate” (Alexis King, 2019).

Tyypit

1 / 3 · Sopimus koodin sisällä

```ts
type Money   = number & { __brand: 'Money' };
type Percent = number & { __brand: 'Percent' };

function percent(value: number): Percent | null {
  return value >= 0 && value <= 100
    ? (value as Percent)
    : null;
}

declare function applyDiscount(
  price: Money,
  discount: Percent,
): Money;
applyDiscount(price, 20);
// ✗ 'number' is not assignable to 'Percent'
```

Yksinkertaistettu esimerkki

Miksi agentille?

- **Domain-tyypit ohjaavat toteutusta.** `Money` ja `Percent` kertovat, mitä luku tarkoittaa.
- **Yksi tie sisään.** `Percent`-arvon saa vain konstruktorista, joka tarkistaa rajat.
- **Testit voi kirjoittaa etukäteen.** Ne nojaavat sopimukseen, eivät koodiin.
- **Väärää ei kirjoiteta vahingossa.** Kääntäjä kertoo heti, missä validointi puuttuu.

Ohitus (`as`, `any`) vaatii tahallisen teon, ja sen voi kieltää lint-säännöllä. Tyyppi ei todista, että validointi on oikein.

Älä pyydä agenttia muistamaan. **Tee unohtamisesta virhe.**

Ominaisuustestit

2 / 3 · Esimerkki vai laki?

Yksi tapaus: tavallinen testi	Kaikki tapaukset: ominaisuustesti
`2 + 3 = 3 + 2`	`a + b = b + a` kaikilla luvuilla `a`, `b`
`sort(sort([3, 1, 2])) = sort([3, 1, 2])`	`sort(sort(xs)) = sort(xs)` kaikilla listoilla `xs`
0 %:n alennus hinnasta 100 € → 100 €	Nollan prosentin alennus **ei koskaan** muuta hintaa

Reunatapaukset. Ominaisuustestit löytävät usein reunatapauksia, jotka eivät käy agentin tai ohjelmistokehittäjän mielessä.

Esimerkki näyttää **yhden tapauksen**. Laki sanoo, mikä pätee **kaikille**.

Ominaisuustestit

2 / 3 · Laki koodina

```
test('0 % discount never changes the price', () => {
  fc.assert(fc.property(anyMoney, (price) =>
    applyDiscount(price, percent(0)!) === price,
  ));
});
```

VIRHEELLINEN TOTEUTUS

```
const DEFAULT_PERCENT = percent(10)!;

function applyDiscount(price: Money, d: Percent) {
  const p = d || DEFAULT_PERCENT; // ✗ 0 on epätosi
  return money(Math.floor(price * (100 - p) / 100));
}
```

Yksinkertaistettu esimerkki · fast-check

HAVAINNOLLISTUS – EI OIKEAN AJON TULOSTE

```
✗ 0 % discount never changes the price
  Property failed after 12 tests
  Shrunk 4 time(s)
  Counterexample:
    price = 1
  Expected 1, got 0
```

Otos, ei todistus: tuhat läpäistyä tapausta ei takaa, ettei vastaesimerkkiä ole.

Vastaesimerkki on agentille **tarkkaa palautetta**: mikä laki, millä syötteellä, mikä tulos.

Hyväksymistestit

3 / 3 · Hyväksymiskriteerit kirjoitetaan jo nyt

SHOP-42 **Story** TYÖN ALLA

Kampanjakoodi kassalla

Asiakkaana haluan syöttää kampanjakoodin kassalla, jotta saan mainostetun alennuksen.

HYVÄKSYMISKRITEERIT

1. Voimassa oleva koodi alentaa maksettavaa summaa koodin prosentin verran.
2. Vanhentunut koodi hylätään, eikä summa muutu.

Kriteerit ovat jo olemassa. Hyväksymistesti tekee niistä **ajettavia**.

Hyväksymistestit

3 / 3 · Täyttyykö tarve

```
Scenario: A valid code lowers the price
  Given the cart total is 100.00
  When the customer enters the code "AUTUMN20"
  Then the amount to pay is 80.00

Scenario: An expired code is rejected
  Given the cart total is 100.00
  And the code "SUMMER10" has expired
  When the customer enters the code "SUMMER10"
  Then the code is rejected
  And the amount to pay is 100.00
```

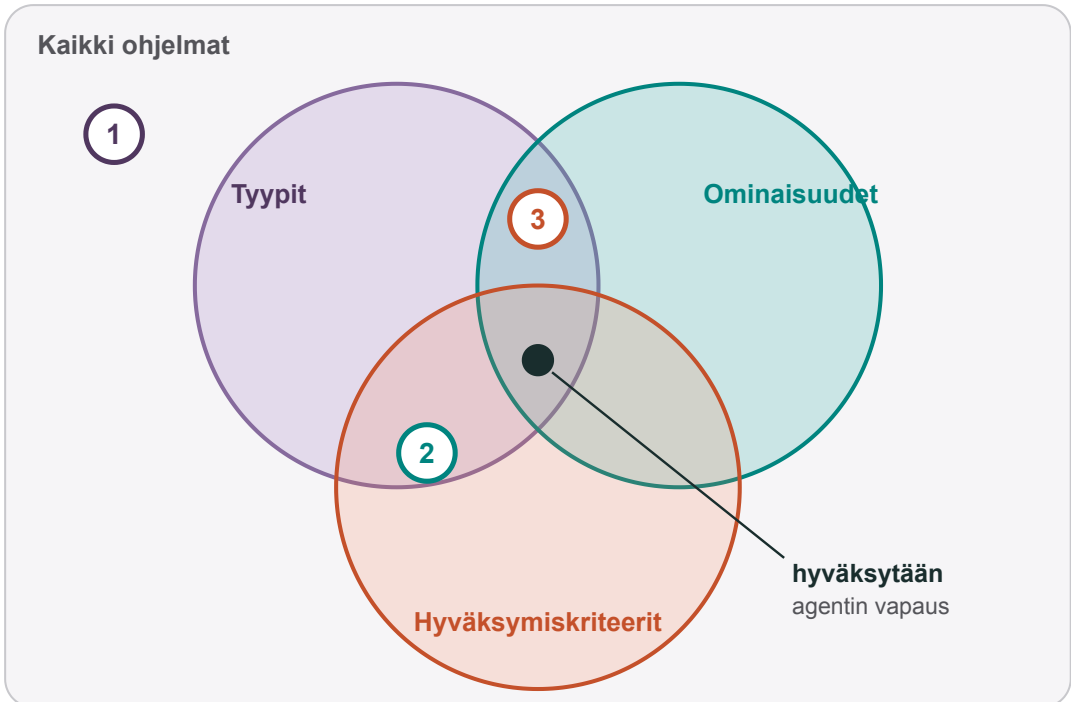
Yksinkertaistettu esimerkki · Gherkin (BDD)

Miksi agentille?

- **Tarpeen kielellä.** Tuoteomistaja voi hyväksyä skenaarion ennen koodia.
- **Agentti ei kirjoita omaa tavoitettaan.** Se pienentää riskiä, että sama väärinkäsitys päätyy koodiin ja testiin.
- **Esimerkki, ei laki.** Skenaario tarkistaa vain nämä luvut.

Hyväksymistesti kertoo, milloin työ on valmis **sen mielestä, joka sitä tarvitsi**.

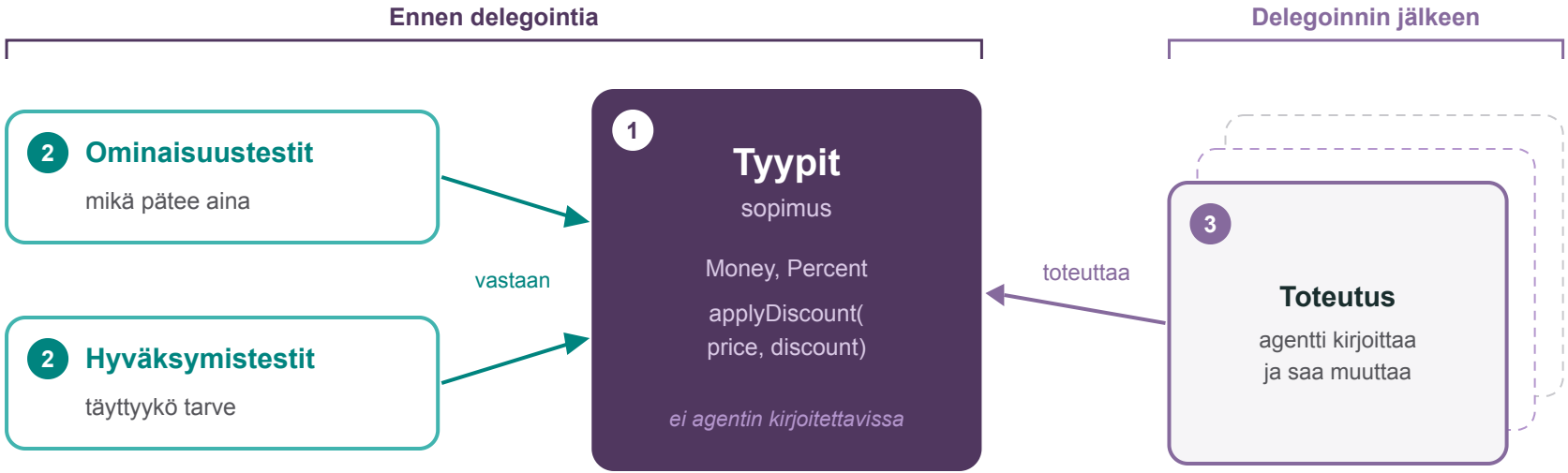
Mikä jää leikkauksen ulkopuolelle?



1. **Ei käänny.** Väärä arvo, kuten 120 %:n alennus.
2. **Laki kaatuu.** 0 %:n alennus muuttaa hintaa.
3. **Skenaario kaatuu.** Alennusta ei tule lainkaan.

Yksikään tarkistus ei riitä yksin. Hyväksytään vain **leikkaus**.

Vakaa rajapinta ensin



1 Tyypit ensin → **2** Testit sopimusta vastaan → **3** Agentti toteuttaa

Tyypit antavat testeille **vakaan rajapinnan**, joten testit voi kirjoittaa ennen kuin toteutusta on olemassa.

Mistä tarkistukset tulevat?

Kuka kirjoittaa, ja mistä?

Ylhäältä alas: epämääräisestä tarpeesta yksityiskohtiin.

HYVÄKSYMISKRITEERIT

Lähde: tiketti ja vaatimusmäärittely

- Tuoteomistaja ja tiimi sopivat
- Mitä tarpeen omistaja odottaa näkevänsä
- Kirjoitetaan jo nyt, kuten dialla 12

OMINAISUUDET

Lähde: insinöörin päättely

- Abstrakteja, teknisiä lakeja: invariantit, idempotenssi, monotonisuus
- Lait, joita kukaan ei kirjoittanut tikettiin, mutta joihin kaikki luottavat

TYYPIT

Lähde: ohjelmoija

- **Turvallisuus rakenteella:** mahdottomat tilat mahdottomiksi esittää
- **Vakaa rajapinta**, jota vastaan testit kirjoitetaan

Kaikki kolme päätetään **ennen** kuin agentti aloittaa toteutuksen.

Mistä tarkistukset tulevat?

Hyväksymiskriteerit · Tiketistä ja vaatimusmäärittelystä

VAATIMUS	SÄÄNTÖ	SÄÄNTÖ	AVOIN KYSYMYS
”Kampanjakoodit toimivat kassalla.” →	**Voimassa oleva koodi alentaa summaa koodin prosentin verran**	**Vanhentunut koodi hylätään, eikä summa muutu**	Pyöristetäänkö alas vai lähimpään senttiin?
	100 € + AUTUMN20 → 80 €	100 € + SUMMER10 (vanhentunut) → hylätään, 100 €	Voiko kahta koodia käyttää yhtä aikaa?
	0,97 € + AUTUMN20 → 0,77 € vai 0,78 €?		

Tekniikka: Example Mapping (Matt Wynne). Sääntö, konkreettiset esimerkit ja avoimet kysymykset.

Konkreettiset esimerkit paljastavat kysymykset **ennen** kuin agentti joutuu arvaamaan.

Mistä tarkistukset tulevat?

Ominaisuudet · Abstraktit lait insinöörin päättelystä

INVARIANTTI

Alennettu hinta on **aina** nollan ja alkuperäisen välissä.

$0 \le \text{apply}(p, d) \le p$

INVARIANTTI

Summa on **aina** kokonaisia senttejä.

$\text{total}(\text{cart}) \in \mathbb{N}$ senttiä

IDEMPOTENSSI

Saman koodin syöttäminen uudelleen ei muuta mitään. *Tämä on päätös.*

$\text{use}(\text{use}(c, k), k) = \text{use}(c, k)$

MONOTONISUUS

Suurempi alennus ei **koskaan** anna suurempaa hintaa.

$d_1 \le d_2 \Rightarrow \text{apply}(p, d_2) \le \text{apply}(p, d_1)$

JÄRJESTYSRIIPPUMATTOMUUS

Tuotteiden järjestys korissa ei vaikuta summaan.

$\text{total}(\text{shuffle}(\text{items})) = \text{total}(\text{items})$

EDESTAKAISUUS

Uuden koodin lisääminen ja poistaminen palauttaa korin ennalleen.

$k \notin c \Rightarrow \text{remove}(\text{use}(c, k), k) = c$

Kriteeri kertoo, mitä asiakas **näkee**. Ominaisuus kertoo, mitä järjestelmä **ei koskaan riko**.

Mistä tarkistukset tulevat?

Tyypit · Mahdottomat tilat mahdottomiksi

✗ SALLII MAHDOTTOMAT TILAT

```
type Campaign = {
  code: string;
  percent: number;
  expired: boolean;
  expiredAt?: Date;
};

// Kääntyy: tyhjä koodi, 120 %, ei päivää
const c: Campaign =
  { code: '', percent: 120, expired: true };
```

✓ MAHDOTON TILA EI KÄÄNNY

```
type Campaign =
  | { status: 'active';
      code: Code; percent: Percent }
  | { status: 'expired';
      code: Code; expiredAt: Date };

declare function applyCode(
  price: Money,
  campaign: Campaign,
): Money;
```

Yksinkertaistettu esimerkki · Code ja Percent ovat brändättyjä tyyppejä kuten dialla 9

Mitä ei voi esittää, sitä **agentti ei kirjoita vahingossa**. Rajapinta pysyy, toteutus saa vaihtua.

Määrittele järjestelmä, älä toteutusta.

Tyypit, ominaisuudet ja hyväksymiskriteerit kertovat, mitä järjestelmä lupaa. **Miten** se tehdään, jää agentille.

Eristä agentti tarkistuksesta

1 / 3 · Agentin työtila

```
checkout/
├── README.md                vaatimukset
├── package.json             skriptit
├── tsconfig.json            kääntäjä
├── src/
│   ├── contracts.ts         tyypit ja rajapinnat
│   ├── constructors.ts      percent(), money()
│   └── implementation/      ✎ kirjoitus
│       └── apply-code.ts
├── test/
│   ├── properties.test.ts   ominaisuustestit
│   └── features/            hyväksymistestit
│       ├── campaign-codes.feature
│       └── steps.ts
└── dist/                    ✎ kirjoitus
```

Demorepon rakenne alennusesimerkin nimillä · kaikki muu kuin ✎: vain luku

```
# Nimetty käyttöoikeusprofiili (Codex CLI)
approval_policy = "never"

[permissions.demo.filesystem]
":root" = "read"

[permissions.demo.filesystem.":workspace_roots"]
"." = "read"
"src/implementation" = "write"
"dist" = "write"

[permissions.demo.network]
enabled = false
```

Demorepon profiili, muotoiltu uudelleen

Agentti saa muuttaa toteutusta vapaasti, mutta **ei sopimusta**.

Eristä agentti tarkistuksesta

2 / 3 · Ohje agentille

Tehtävä: Toteuta **SHOP-42 ”Kampanjakoodi kassalla”**. Vaatimukset ovat tiedostossa `README.md` ja sopimukset tiedostossa `src/contracts.ts`.

SÄÄNNÖT

1. Aja ensin käännös ja testit, ja raportoi lähtötilan virheet.
2. Toteuta olemassa olevia sopimuksia vasten. Käytä tarkistusten palautetta korjaamiseen.
3. **Älä heikennä, poista, ohita tai muuta** tarkistuksia, sopimuksia tai vaatimuksia saadaksesi testit läpi.
4. Jos tarkistus on mielestäsi väärä, **pysähdy ja merkitse se ihmisen arvioitavaksi**.
5. Operaattori ajaa tarkistukset **uudelleen itsenäisesti**.
6. Kuvaa jäljellä olevat riskit ja näytön rajat.

Lyhennetty ja suomennettu demorepon kehotteesta · tehtävärivi mukautettu esimerkkiin

Ohje kertoo säännöt. **Työtila ja tarkistus** pakottavat ne.

Eristä agentti tarkistuksesta

3 / 3 · Mitä asetelma ratkaisee

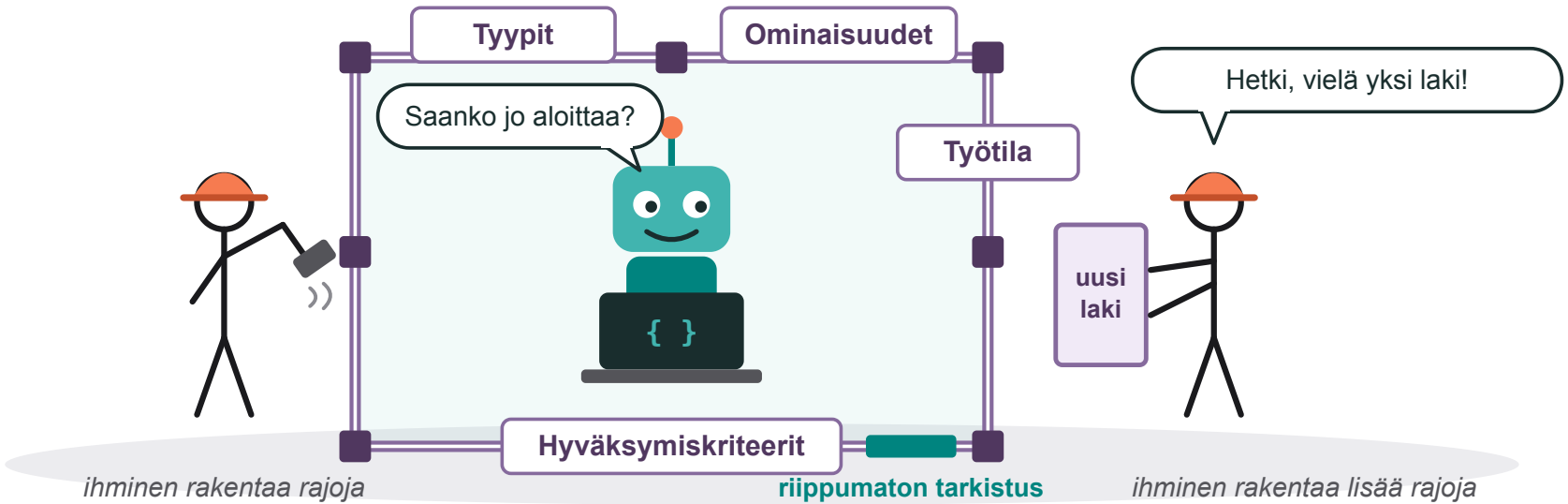
1	Sama väärinkäsitys kahdesti	→ **Agentti ei kirjoita tarkistuksia** Ihminen päättää ne ennen delegointia
2	Testejä voi muuttaa	→ **Testit ovat vain luku -tilassa** Käyttöoikeusprofiili lukitsee ne
3	Agentti erehtyy valmiudestaan	→ **Tarkistukset ajetaan uudelleen** Puhtaassa kopiossa, kuten CI:ssä. Agentilta tulee mukaan vain toteutus.

Ongelmat dialta 3

Valmis = agentti lopetti **ja** riippumaton tarkistus meni läpi. Agentin oma ”testit menevät läpi” ei ratkaise.

Missä ihmisen työ on?

Rakennustelineet ennen rakennusta

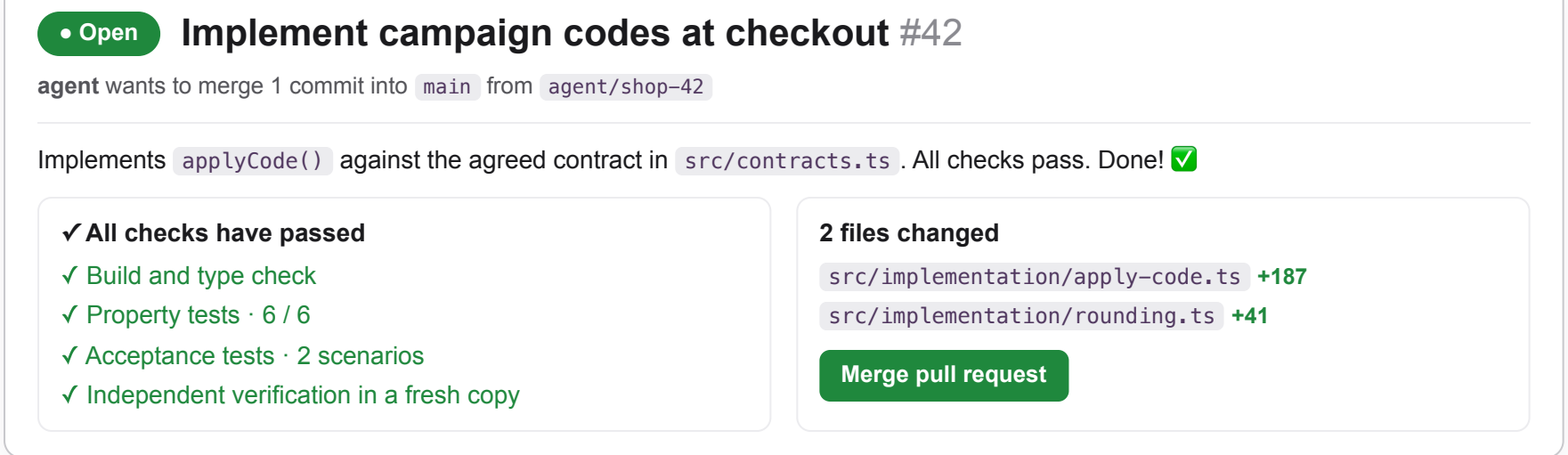


Suurin osa ihmisen työstä ja huomiosta menee **tarkistusten rakentamiseen**. Agentin työ on kaikki aitauksen sisällä.

Päästä agentti irti

Aitauksen sisällä agentti saa tehdä mitä haluaa. Ulos pääsee vain tarkistuksen portista.

HAVAINNOLLISTUS – KEKSITTY PR



Mergeäisitkö tämän lukematta diffiä?

Mitä vihreä tulos ei kerro?

Ovatko vaatimukset oikeat?

Tarkistukset tarkistavat sen, mitä kirjoitimme. Eivät sitä, mitä olisi pitänyt kirjoittaa.

Mitä ei ole katettu?

Mitä sopimus ja lait eivät mainitse, sitä ei ole tarkistettu. Generaattorit tuottavat vain sen, mitä niille kerrottiin.

Luotammeko suoritukseen?

Ehdokaskoodi ajetaan samassa prosessissa kuin testit. Kielletyt pakotiet, kuten `as` ja `any`, nojaavat ohjeeseen ja katselmointiin.

Onko tämä tuotantoa?

Vihreä kertoo, että koodatut odotukset täyttyivät. Ei sitä, että järjestelmä on valmis tuotantoon.

Onko koodi ylläpidettävää?

Onko se helppo ymmärtää ja muuttaa? Seuraako se tiimin tapoja? Toimiiko se riittävän nopeasti?

Onko ratkaisu järkevä?

Onko se turhan monimutkainen? Toiko se uusia riippuvuuksia? Sopiiko se muuhun arkkitehtuuriin?

Tarkistukset eivät poista katselmointia. Ne kertovat, **mihin** katselmointi kannattaa käyttää.

Kuinka paljon huomiota muutos ansaitsee?

	VÄHEMMÄN HUOMIOTA	ENEMMÄN HUOMIOTA
Verifioitavuus	tarkistukset kattavat muutoksen	ei koneellisesti tarkistettavissa
Vaikutusalue	yksi sisäinen moduuli	maksut, tietoturva, data
Peruttavuus	yksi revert, feature flag	migraatio, lähetetty viesti
Uutuus	toistuva, tuttu muutos	uusi arkkitehtuuripäätös

Mitoita ihmisen huomio **muutoksen** mukaan, ei sen mukaan, kuka sen kirjoitti.

Kolme askelta



1. Määrittele järjestelmä, älä toteutusta
2. Eristä agentti tarkistuksesta
3. Päästä agentti irti

Kiitos! Kysymyksiä?

LOIHDE